

Trabalho Máquina de Estados

1. Visão Geral do Cenário

O cenário é um restaurante, onde temos o agente cozinheiro e o agente cliente. Como um restaurante usual, o cliente fará pedidos com base na sua fome, e o cozinheiro terá que anotar os pedidos e cozinhar com base no que foi pedido, e quando o preparo estiver pronto, entregará ao cliente que comerá o pedido, formando um ciclo.

2. Visão Geral de Cada Agente

- Cliente

O cliente sente fome e faz pedidos com base no limiar dela, e ao receber comida, dependendo de qual tiver sido pedida, restaura mais ou menos pontos de fome. O limiar de fome diminui com base no tempo, repetindo o ciclo.

- Cozinheiro

O cozinheiro depende de que o cliente faça o pedido para começar a cozinhar. Dependendo do que foi pedido, o tempo de preparo é menor ou maior. Ao terminar de preparar o pedido, ele entrega ao cliente.

3. Diagrama de Estados por Agente

(Imagem inserida no documento)

4. Tabela de Regras por Agente

Agente Cozinheiro

- > Anota Pedido
- > Cozinha
- > Entrega pedido

Só se pode anotar pedido se um pedido do agente B for feito

Só pode cozinhar se anotar pedido

Só pode entregar pedido se a comida estiver pronta (10s de preparo Feijoadada, 5s hambúrguer).

Agente Cliente

- > Faz pedido (Feijoadada ou Hambúrguer)
- > Come
- > Ficar com fome

Só faz pedido se estiver com fome
Só pode comer se o estado "entregar pedido" estiver concluído
Só pode fazer pedido novo após terminar de comer
Comer o pedido diminui a fome
Dependendo do pedido de feijoadá ou hambúrguer, a quantidade de fome restaurada é maior ou menor

5. Descrições das variáveis/limiares por agentes

Agente Cliente:

'fome' -> Representa o nível de fome do cliente
'tempoComendo' -> Conta por quanto tempo o cliente fica no estado 'Come'
'restaurante' -> Permite o cliente criar e consultar o pedido atual

Estado inicial - 'FicaComFome': Cliente começa acumulando fome.
Limiar de fome - '>= 10': Ao chegar em 10, o cliente faz um pedido.
Tempo pra comer - '>= 3': Após 3 ciclos comendo, termina de comer a refeição.

Agente Cozinheiro:

'tempoPreparo': Tempo acumulado de preparação do prato.
'restaurante': Permite acessar o mesmo pedido criado pelo cliente.

Estado inicial - 'AnotaPedido': Aguarda a existência de um novo pedido.
Limiar de preparo - (o valor inicial depende do prato escolhido): Define quando o pedido fica pronto.

6. Estrutura do Código

'Character'

Representa os agentes da simulação
Possui:
- printStats()
- update()

As duas principais classes que implementam nesse script são **Cliente** e **Cozinheiro**

'State'

Define o contrato obrigatório dos estados.
Cada estado pode possuir:
- enter()
- execute()

- leave()

‘AbstractState’

Sua função é guardar uma ref ao personagem/agente relacionado ao estado:

‘ AbstractState<Cliente> ’

‘ AbstractState<Cozinheiro> ’

‘Cliente’

Cliente

```
]
+- - FicaComFome
]
+- - FazPedido
]
+- - Come
```

‘FicaComFome’

- aumenta a fome.
- verifica o limiar ‘ fome >= 10 ‘.
- inicia um pedido.

‘FazPedido’

- seleciona um prato aleatório.
- criar ‘ Pedido ‘.
- aguardar o pedido ser entregue.

‘Come’

- simular o tempo de alimentação.
- terminar a refeição após 3 ciclos.
- zerar a fome.
- limpar o pedido.

‘Cozinheiro’

Cozinheiro

```
]
+- - AnotaPedido
]
+- - Cozinha
]
+- - EntregaPedido
```

‘AnotaPedido’

Espera um pedido com o status ‘ FEITO ’

Depois:

‘ FEITO ’ -> ‘ ANOTADO ’
então entra na ‘Cozinha’

‘Cozinha’

Obtém o tempo necessário de acordo com o prato, incrementa ‘ tempoPreparo ’ e, quando o limite é atingido:

‘ ANOTADO ’ -> ‘ PRONTO ’

‘EntregaPedido’

Realiza:

‘ PRONTO ’ -> ‘ ENTREGUE ’

e volta pra ‘ AnotaPedido ’.

‘Restaurante’

É o ponto de comunicação entre ‘ Cliente ’ e ‘ Cozinheiro ’.
Apenas possui:

Pedido pedidoAtual

e métodos para:

- consultar o pedido.
- alterar o pedido
- verifica se existe um pedido

O ‘ Cliente ’ e ‘ Cozinheiro ’ não se comunicam diretamente, o cliente se comunica com o ‘ Restaurante ’. O cozinheiro, por sua vez, apenas verifica ‘ pedidoAtual ’.

‘State Machine’

A simulação começa em ‘ StateMachine ’.
Ela:

- cria um ‘ Restaurante ’.
- cria um ‘ Cliente ’.
- cria um ‘ Cozinheiro ’
- adiciona os dois em uma lista de ‘ Character ’.
- executa ‘ update() ’ continuamente.
- espera 1 segundo
- repete indefinidamente.

7. Resultados esperados

Cliente sente fome: a fome aumenta gradualmente até atingir o valor 10.

Cliente faz o pedido: ao atingir o limite de fome, o cliente escolhe um prato de maneira aleatória e cria um novo pedido

Cozinheiro anota o pedido: o cozinheiro identifica o novo pedido e altera seu status de ' FEITO ' para ' ANOTADO '.

Cozinheiro prepara o prato: o tempo de preparo aumenta a cada ciclo até atingir o tempo definido do prato escolhido.

Pedido fica pronto: ao finalizar o preparo, o status do pedido é alterado para ' PRONTO '.

Pedido é entregue: o cozinheiro entrega o pedido e altera seu status pra ' ENTREGUE '.

Cliente come: após receber o pedido, o cliente permanece comendo durante 3 ciclos

Cliente termina a refeição: a fome é zerada, o pedido atual é removido e o cliente volta a sentir fome.

Repetição do Processo: o sistema continua executando esse ciclo continuamente enquanto o programa estiver aberto.

8. Limitações

Pensamos em adicionar um estado adicional de sede, que seguiria o mesmo padrão de fome, porém seria repetitivo e mais do mesmo, então decidimos contra.

9. Referências

A inteligência artificial Claude foi utilizada para ajudar a criar o README.md.